Build: Todo App with CRUD

Problem: Build a todo app with add, toggle complete, edit, and delete. Include a filter (all/active/completed).

What This Tests: Full CRUD operations, filtering, immutable state updates, useReducer.

jsx

JSX Copy

```jsx
function todoReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return [...state, { id: Date.now(), text: action.text, done: false }
    case 'TOGGLE':
      return state.map(t => t.id === action.id ? { ...t, done: !t.done } :
    case 'DELETE':
      return state.filter(t => t.id !== action.id);
    case 'EDIT':
      return state.map(t => t.id === action.id ? { ...t, text: action.text
    default:
      return state;
  }
}

function TodoApp() {
  const [todos, dispatch] = useReducer(todoReducer, []);
  const [input, setInput] = useState('');
  const [filter, setFilter] = useState('all');
  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState('');

  const handleAdd = () => {
    if (!input.trim()) return;
```

```
  dispatch({ type: 'ADD', text: input.trim() });
  setInput('');
};

const handleKeyDown = (e) => {
  if (e.key === 'Enter') handleAdd();
};

const startEdit = (todo) => {
  setEditingId(todo.id);
  setEditText(todo.text);
};

const saveEdit = (id) => {
  if (editText.trim()) {
    dispatch({ type: 'EDIT', id, text: editText.trim() });
  }
  setEditingId(null);
};

const filtered = todos.filter(todo => {
  if (filter === 'active') return !todo.done;
  if (filter === 'completed') return todo.done;
  return true;
});

return (
  <div>
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} onKe
      <button onClick={handleAdd} disabled={!input.trim()}>Add</button>
    </div>

    <div>
      {['all', 'active', 'completed'].map(f => (
        <button key={f} onClick={() => setFilter(f)} style={{ fontWeight
          {f.charAt(0).toUpperCase() + f.slice(1)}
        </button>
      ))}
    </div>

    {filtered.length === 0 ? (
      <p>{filter === 'all' ? 'No todos yet. Add one above!' : `No ${filt
    ) : (
```

```
            {editingId === todo.id ? (
              <>
                <input value={editText} onChange={e => setEditText(e.tar
                <button onClick={() => saveEdit(todo.id)}>Save</button>
                <button onClick={() => setEditingId(null)}>Cancel</butto
              </>
            ) : (
              <>
                <span onClick={() => dispatch({ type: 'TOGGLE', id: todo
                  {todo.text}
                </span>
                <button onClick={() => startEdit(todo)}>Edit</button>
                <button onClick={() => dispatch({ type: 'DELETE', id: to
              </>
            )}
          </li>
        ))}
      </ul>
    )}

    <p>{todos.filter(t => !t.done).length} items remaining</p>
  </div>
 );
}
```

Key Concepts: useReducer for CRUD, inline editing with separate edit state, filtering with derived state, Enter key handling, empty state, disabled button for empty input.

QUICK CHECK — no marks, just for you.

1. In this todo app, how are the add, toggle, edit, and delete operations on the todo list managed?

- ○ A useMemo that recalculates the todo array from the input field on every render
- ○ A useContext provider that exposes the todo array and lets each list item write to it directly

- [ ] A single useState array that each handler mutates in place with push, splice, and direct property assignment
- [ ] A todoReducer wired up with useReducer, where each handler dispatches a typed action such as ADD or TOGGLE

Check answer

2. When the user clicks the "Active" filter button, what actually happens to the todos?

- [ ] The reducer receives a FILTER action that replaces the stored todo array with only the active todos
- [ ] The completed todos are dispatched through DELETE so that only active ones remain in state
- [ ] The todos array is sorted so completed items move to the bottom and are hidden with CSS
- [ ] Nothing is removed from state; the filter value changes and a filtered list is derived from todos during render

Check answer

3. The user clicks Edit on a todo, types new text, then clicks Cancel. Why is the original todo text still intact?

- [ ] Each keystroke dispatches EDIT, but Cancel re-dispatches EDIT with the text captured when editing started
- [ ] The reducer keeps a history stack, so Cancel dispatches an UNDO action that restores the previous text
- [ ] Typing only updates a separate editText state; the EDIT action is dispatched from saveEdit, and Cancel just clears editingId

The todo objects are frozen while editingId is set, so no dispatch can change their text until Save runs

Check answer

Previous
Build: Dynamic Checkbox with Select All

Next
Build: Search Bar with Debounced API Calls

LEARN MORE

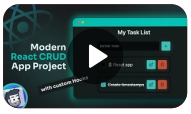
ARTICLES & DOCS

Updating Arrays in State
react.dev

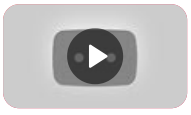
VIDEOS



React todo list (CRUD)
YouTube



Modern React CRUD todo app
YouTube



Todo app from scratch (hooks)
YouTube

Hand-picked references for this lesson.